

Data Structures and Algorithms

GÉZA HORVÁTH

Sixth Lecture

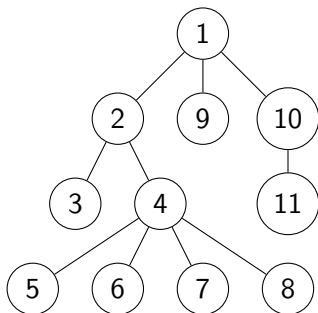
Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

The tree – abstract data type

A tree consists of a set of nodes and a set of edges (connecting pairs of nodes), satisfying the following properties:

- A tree has the property that there is exactly one path (no more, no less) between any pair of nodes, where a path is a connected sequence of edges.
- In a rooted tree, one distinguished node is called the root.



The tree – important expressions

- node
 - root
 - internal node (root included)
 - leaf
- edge: connecting pairs of nodes
- parent
- ancestor: parent, grandparent, grand-grandparent, etc.
- depth of a node: number of ancestors
- height of a tree: maximum depth of any node
- child
- descendant: child, grandchild, grand-grandchild, etc.
- sibling: two nodes that have the same parent
- path: connected sequence of edges
- subtree

The tree – data structure

Tree properties:

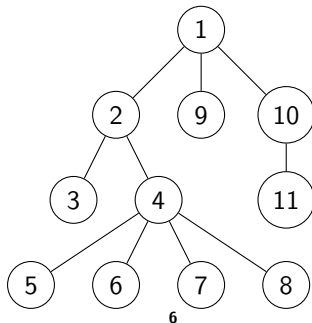
- solid
- hierarchical
- dynamic
- sometimes continuously represented, sometimes represented by pointers

The tree – linked representation

We are unable to represent the tree with the following data for nodes:

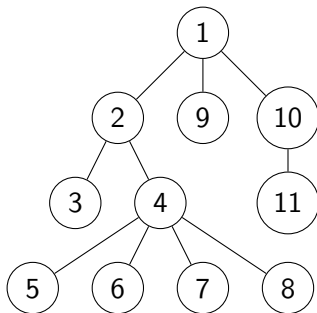
- value, plus
- pointers for each child of the node,

because we do not know at the beginning, how many pointers we need in the future, when we add nodes to the tree. Instead, we use exactly 2 pointers for each node, one is for the leftmost child and one is for the right sibling of the node. (See the drawing on the whiteboard!)



The tree – continuous representation

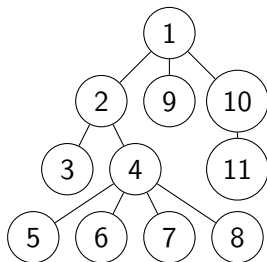
One simple possibility for continuous tree representation is hierarchical list. (Unfortunately we lose direct access.)



(1(2(3)(4(5)(6)(7)(8)))(9)(10(11))))

The tree – traversal

In computer science, tree traversal (also known as tree search) is a form of graph traversal and refers to the process of visiting (checking and/or updating) each node in a tree exactly once. Such traversals are classified by the order in which the nodes are visited.

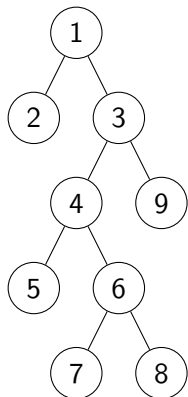


Preorder: 1,2,3,4,5,6,7,8,9,10,11

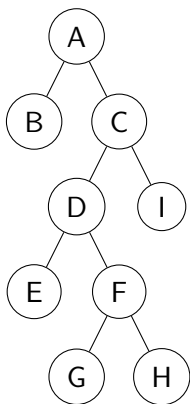
Postorder: 3,5,6,7,8,4,2,9,11,10,1

Binary tree – abstract data type

In computer science, a binary tree is a tree data structure in which each node has at most two children, which are referred to as the left child and the right child.



Binary tree – linked representation



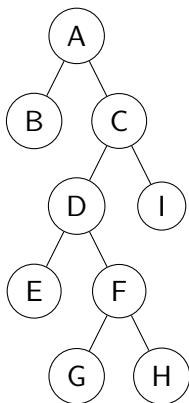
In the case of a binary tree, we are able to represent the tree

with the following data for each node:

- value, plus
- pointers for each child of the node,

because we do know at the beginning, that we need maximum two pointers in the future. So we use exactly 2 pointers for each node, one is for the left child and one is for the right child of the node. (See the drawing on the whiteboard!)

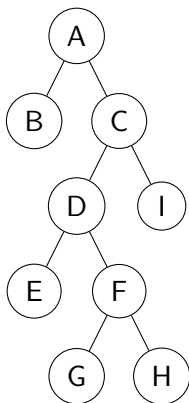
Binary tree – continuous representation



	key	left	right
1	A	2	3
2	B	0	0
3	C	4	9
4	D	5	6
5	E	0	0
6	F	7	8
7	G	0	0
8	H	0	0
9	I	0	0

Root: 1

Binary tree – continuous representation

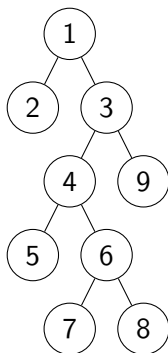


	key	left	right	parent
1	A	2	3	0
2	B	0	0	1
3	C	4	9	1
4	D	5	6	3
5	E	0	0	4
6	F	7	8	4
7	G	0	0	6
8	H	0	0	6
9	I	0	0	3

Root: 1

Binary tree – traversal

In computer science, tree traversal (also known as tree search) is a form of graph traversal and refers to the process of visiting (checking and/or updating) each node in a tree exactly once. Such traversals are classified by the order in which the nodes are visited.



Preorder: 1,2,3,4,5,6,7,8,9

Inorder: 2,1,5,4,7,6,8,3,9

Postorder: 2,5,7,8,6,4,9,3,1

We **can not reconstruct** the binary tree from preorder, inorder or postorder traversal.

Binary tree – traversal

PREORDER-TREE-TRAVERSAL(x)

- 1 if $x \neq \text{NIL}$
- 2 print $x.\text{key}$
- 3 PREORDER-TREE-TRAVERSAL($x.\text{left}$)
- 4 PREORDER-TREE-TRAVERSAL($x.\text{right}$)

INORDER-TREE-TRAVERSAL(x)

- 1 if $x \neq \text{NIL}$
- 2 INORDER-TREE-TRAVERSAL($x.\text{left}$)
- 3 print $x.\text{key}$
- 4 INORDER-TREE-TRAVERSAL($x.\text{right}$)

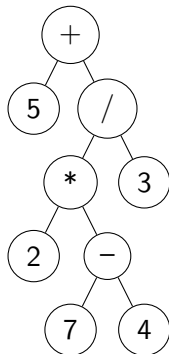
POSTORDER-TREE-TRAVERSAL(x)

- 1 if $x \neq \text{NIL}$
- 2 POSTORDER-TREE-TRAVERSAL($x.\text{left}$)
- 3 POSTORDER-TREE-TRAVERSAL($x.\text{right}$)
- 4 print $x.\text{key}$

Expression tree – abstract data type

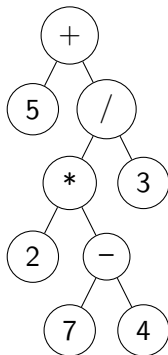
Expression tree is a binary tree in which each internal node corresponds to operator and each leaf node corresponds to operand.

Example: $5+2*(7-4)/3$



Expression tree – traversal

Example: $5+2*(7-4)/3$



Prefix: $+5/*2-743$

Infix: $5+2*7-4/3$

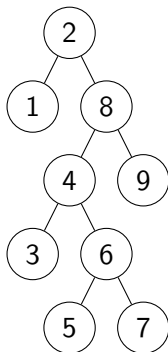
Postfix: $5274-*3/+$

We can reconstruct the expression tree from prefix expression and postfix expression.

Binary search tree – abstract data type

Binary search tree is a binary tree satisfying the binary-search-tree property:

Let x be a node in a binary search tree. If y is a node in the left subtree of x , then $y.\text{key} < x.\text{key}$. If y is a node in the right subtree of x , then $y.\text{key} > x.\text{key}$.



Binary search tree – operations

- **Search**
- **Insert**
- **Delete**
- Minimum
- Maximum
- Predecessor
- Successor

Basic operations on a binary search tree take time proportional to the height of the tree. For a complete binary tree with n nodes, such operations run in $\Theta(\log_2 n)$ worst-case time. If the tree is a linear chain of n nodes, however, the same operations take $\Theta(n)$ worst-case time.

Binary search tree – operations

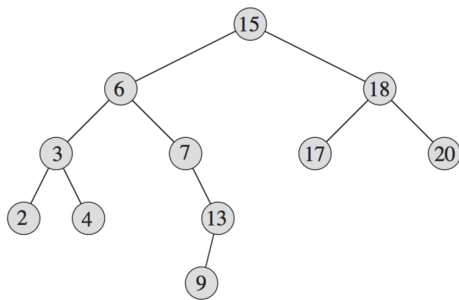


Figure 12.2 Queries on a binary search tree. To search for the key 13 in the tree, we follow the path $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$ from the root. The minimum key in the tree is 2, which is found by following *left* pointers from the root. The maximum key 20 is found by following *right* pointers from the root. The successor of the node with key 15 is the node with key 17, since it is the minimum key in the right subtree of 15. The node with key 13 has no right subtree, and thus its successor is its lowest ancestor whose left child is also an ancestor. In this case, the node with key 15 is its successor.

Binary search tree – operations

TREE-SEARCH(x, k)

```

1  if  $x == \text{NIL}$  or  $k == x.\text{key}$ 
2      return  $x$ 
3  if  $k < x.\text{key}$ 
4      return TREE-SEARCH( $x.\text{left}, k$ )
5  else return TREE-SEARCH( $x.\text{right}, k$ )

```

We can rewrite this procedure in an iterative fashion by "unrolling" the recursion into a while loop. (The iterative version is more efficient.)

ITERATIVE-TREE-SEARCH(x, k)

```

1  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$ 
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else  $x = x.\text{right}$ 
5  return  $x$ 

```

Binary search tree – operations

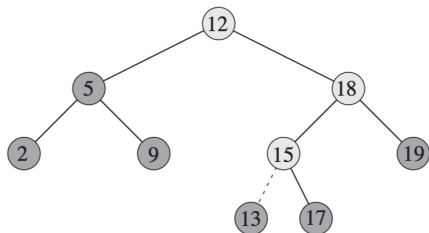


Figure 12.3 Inserting an item with key 13 into a binary search tree. Lightly shaded nodes indicate the simple path from the root down to the position where the item is inserted. The dashed line indicates the link in the tree that is added to insert the item.

Binary search tree – operations

TREE-INSERT(T, z)

```

1   $y = \text{NIL}$ 
2   $x = T.\text{root}$ 
3  while  $x \neq \text{NIL}$ 
4       $y = x$ 
5      if  $z.\text{key} < x.\text{key}$ 
6           $x = x.\text{left}$ 
7      else  $x = x.\text{right}$ 
8   $z.p = y$ 
9  if  $y == \text{NIL}$ 
10      $T.\text{root} = z$            // tree  $T$  was empty
11 elseif  $z.\text{key} < y.\text{key}$ 
12      $y.\text{left} = z$ 
13 else  $y.\text{right} = z$ 
```

Binary search tree – operations

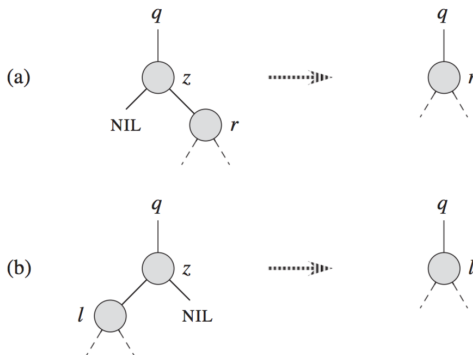
Deletion

The overall strategy for deleting a node z from a binary search tree T has three basic cases but, as we shall see, one of the cases is a bit tricky.

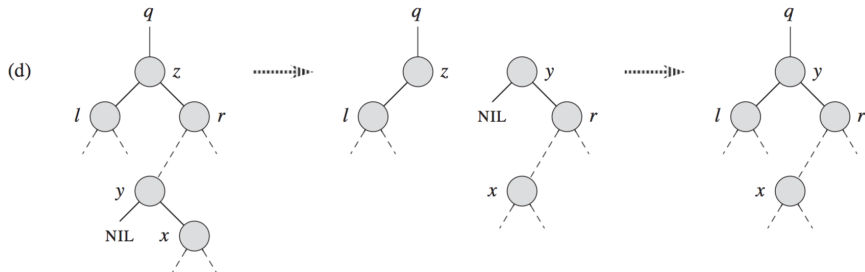
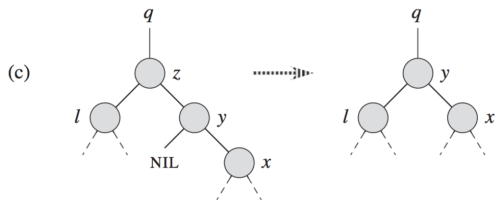
- If z has no children, then we simply remove it by modifying its parent to replace z with NIL as its child.
- If z has just one child, then we elevate that child to take z 's position in the tree by modifying z 's parent to replace z by z 's child.
- If z has two children, then we find z 's successor y —which must be in z 's right subtree—and have y take z 's position in the tree. The rest of z 's original right subtree becomes y 's new right subtree, and z 's left subtree becomes y 's new left subtree. This case is the tricky one because, as we shall see, it matters whether y is z 's right child.

Binary search tree – operations

Figure 12.4 Deleting a node z from a binary search tree. Node z may be the root, a left child of node q , or a right child of q . **(a)** Node z has no left child. We replace z by its right child r , which may or may not be NIL. **(b)** Node z has a left child l but no right child. We replace z by l . **(c)** Node z has two children; its left child is node l , its right child is its successor y , and y 's right child is node x . We replace z by y , updating y 's left child to become l , but leaving x as y 's right child. **(d)** Node z has two children (left child l and right child r), and its successor $y \neq r$ lies within the subtree rooted at r . We replace y by its own right child x , and we set y to be r 's parent. Then, we set y to be q 's child and the parent of l .



Binary search tree – operations

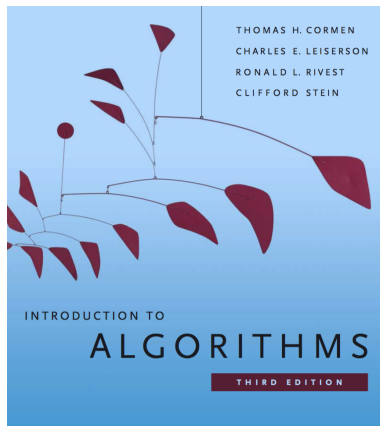


Binary search tree – data structure operations

Implemented data structure operations:

- **create** data structure – **linked or continuous** representation
- **modify** data structure
 - add elements – TREE-INSERT
 - delete elements – TREE-DELETE
 - change elements – **not important**
- **access** – ITERATIVE-TREE-SEARCH

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.